

Proyecto de semestre Azure DevOps-GitHub

Samuel Arturo Daza Criado

Jayber Santiago Castro Parra

Universidad de Cundinamarca sede Fusagasugá

Ingeniería de Software II

Ing. German Ortiz Diaz

2026

Tabla de Contenido

Índice de Figuras'	3
Índice de Tablas	3
Introducción.....	4
Objetivos	4
Objetivo General	4
Objetivos Específicos.....	4
Descripción del problema	5
Metodología de Desarrollo.....	6
Marco de Trabajo	7
Historias de Usuario	8
Product Backlog	10
Sprint.....	11
Modelo de base de datos	13
Modelo entidad relación	13
Diagramas UML	14
Diagrama de casos de uso	14
Diagrama de Clases	15
Desarrollo del Software	15
Pruebas Unitarias	17
Evidencia de instalaciones y registros	18
Plataforma Cloude Azure DevOps	18

Control de Versiones GitHub	19
Enlaces de Apoyo	21
Conclusiones.....	21

Índice de Figuras'

Figura 1	6
Figura 2	11
Figura 3	12
Figura 4	13
Figura 5	14
Figura 6	15
Figura 7.	15
Figura 8	16
Figura 9.	16
Figura 10	17
Figura 11	18
Figura 12	18
Figura 13	19
Figura 14	19
Figura 15	20
Figura 16	20

Índice de Tablas

Tabla 1.....	8
Tabla 2.....	10

Introducción

Este documento detalla los avances y el proceso de desarrollo del software Simus_mjn, documentando de manera integral la planificación, las herramientas empleadas y la metodología aplicada. Asimismo, se describen las actividades realizadas en plataformas de gestión y control de versiones como Azure DevOps y GitHub, presentando como resultado un prototipo funcional que evidencia el diseño y desarrollo inicial del sistema

Objetivos

Objetivo General

Presentar el progreso funcional del software Simus_mjn, integrando evidencias de diseño y desarrollo obtenidas mediante herramientas digitales, para validar el avance del proyecto.

Objetivos Específicos

- Asegurar el cumplimiento de los lineamientos específicos de entrega *en la guía de la actividad*
- Elaborar los esquemas solicitados en la rúbrica dada, *con el fin de visualizar los componentes y el diseño del software Simus_mjn."*
- *Estructurar y cargar los archivos fuente, así como la documentación técnica, en los repositorios de GitHub y Azure DevOps para una entrega organizada y accesible*

Descripción del problema

La Neurodegeneración asociada al gen pantotenato quinasa (PKAN) es una patología huérfana o poco frecuente que afecta el sistema nervioso, manifestándose principalmente en la infancia. Esta condición genera trastornos del movimiento, como distonía y rigidez, lo que limita significativamente la motricidad voluntaria y la coordinación de los pacientes.

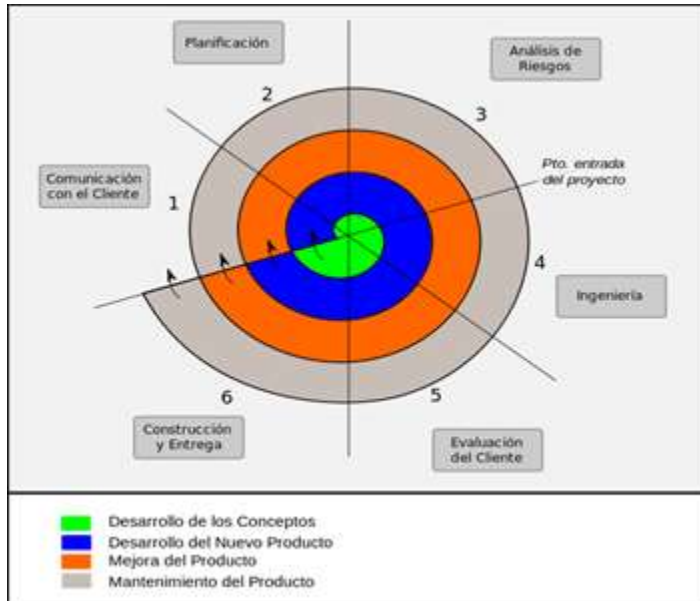
A pesar del auge tecnológico actual, existe una brecha importante en el desarrollo de herramientas de asistencia específicas para esta población. Las soluciones de estimulación motriz disponibles suelen ser genéricas, de alto costo o carecen de adaptaciones que consideren las limitaciones motoras finas de los niños con PKAN, lo que reduce la efectividad de sus procesos de rehabilitación y afecta su calidad de vida.

Ante este panorama, surge la necesidad de desarrollar el software **Simus_mjn**, una solución tecnológica diseñada para incentivar la estimulación de movimientos voluntarios a través de una interfaz interactiva y accesible, adaptada específicamente a los requerimientos neurofisiológicos de estos pacientes.

Metodología de Desarrollo

Figura 1

Modelo Espiral



Nota. En la Figura se puede observar la metodología de desarrollo en espiral y cada una de sus Fases. Autor

El modelo en espiral se acomodó al proyecto, pues requería el uso de tecnologías emergentes, entre esas se tiene el reconocimiento facial por medio de Opencv y uso de API TTS. Estas tecnologías necesitan examinación frecuente, prototipado anticipado y validación directa con el usuario final.

Este modelo se basa en iteraciones, y cada una se estructura en 4 fases principales: la planificación, el análisis de riesgos, la implementación y la evaluación del software. Cada ciclo de la espiral posibilitó la revisión de los requisitos, analizar y eliminar riesgos del sistema, elaborar aumento funcional y validar avances del software, esto con el usuario directo en el uso del software.

Marco de Trabajo

El desarrollo del software SIMUS.MJN opto por el uso del marco de trabajo XP (Extreme Programming) con el objetivo de asegurar la adaptabilidad a los cambios y la satisfacción del usuario final. Dicho marco fue integrado de manera practica en cada fase del proyecto, de la mano con el modelo de desarrollo tradicional en espiral, empleado como base metodológica. Las practicas centrales de este marco de trabajo que se aplicaron en la planificación e ingeniería del software fueron las siguientes: Programación en pareja: Cada parte del código fue implementada y detallada de manera colaborativa por los integrantes del equipo. Esta práctica permitió la detección de errores de manera temprana, adquirir conocimiento y manejar coherencia entre los distintos módulos del software como la detección facial, el control de gestos y los juegos. Desarrollo iterativo con entregas frecuentes: El proyecto fue particionado en pequeñas iteraciones que finalizaban con una versión funcional del software. Cada entrega era puesta a revisión directa con el usuario objetivo (paciente con PKAN), lo que posibilito la obtención de retroalimentación continua sobre la accesibilidad, detección de gestos y respuesta del sistema. Pruebas continuas: Por cada incremento, se llevaron a cabo pruebas unitarias para validar el correcto funcionamiento de los módulos de detección facial, control de cursor, reconocimiento manual y facial, sistema de clics, configuración y juegos. Las pruebas ayudaron a refactorizar el código constantemente, mejorando su estructura sin alterar su función, con el fin de tener un software pulido y estable. Simplicidad y diseño evolutivo: El diseño del software se trabajó lo más simple posible, posibilitando su adaptación a nuevas funciones sin que su estructura presentara complejidad. La interfaz fue manejada de tal 38 forma que fuera intuitiva, con accesos visuales grandes y manejados por cámara, de acuerdo con los principios de accesibilidad de XP. Integración continua: Cada avance o modulo nuevo fue integrado y puesto a prueba inmediatamente dentro del proyecto principal, asegurando la compatibilidad y minimizando errores finalizando el desarrollo. Gracias a la implementación del marco XP, el desarrollo del software SIMUS.MJN tuvo una buena calidad técnica, redujo tiempos de

corrección y mejoro la experiencia del usuario final. En agregado, el trabajo colaborativo permitió la construcción de un software sólido, de fácil acceso y adaptable a las necesidades reales del publico principal.

Historias de Usuario

Tabla 1.

Historias de Usuario

HU-#	ID en Azure DevOps	Historia de Usuario	Prioridad
HU-01	59	Como desarrollador debo eliminar el apartado de colores locos, dado que no está aportando información clave del funcionamiento del software al usuario.	Media-Alta
HU-02	60	Como desarrollador debo añadir más contenido e implementar aleatoriedad al juego animalia para que sea más entretenido y visualmente atractivo	Media
HU-03	61	Como Usuario quiero poder jugar con un límite estipulado, añadiendo más dificultad en cada uno de los mismos	Media
HU-04	54	Como desarrollador requiero estabilizar un solo modo de uso directo para el software que garantice estabilidad en el mismo	Alta
HU-05	53	Como Usuario quiero poder diligenciar mi verdadera fecha de nacimiento en la creación de mi sesión en el software	Media-Alta
HU-06	86	Como usuario quiero poder cerrar la aplicación en el apartado de inicio de sesión de manera intuitiva	Media
HU-07	104	Como usuario quiero ver el mismo cursor en cualquier apartado	Baja
HU-08	108	Como usuario requiero que el software tenga interfaz intuitiva y sin desorden visual.	Baja

HU-09	112	Como usuario quiero que el apartado de instrucciones sea más claro e intuitivo	Baja
HU-10	116	Como desarrollador requiero que el software tenga buen aspecto visual y estabilidad en los cambios de los apartados	Alta
HU-11	120	Como desarrollador necesito que el Text to Speech del software sea claro y conciso para el usuario.	Media
HU-12	124	Como Acompañante necesito poder descargar un pdf con el informe dado de la aplicación con respecto al avance del usuario dentro de la aplicación.	Alta
HU-13	129	Como desarrollador necesito que la información procesada del usuario se pueda reflejar de manera clara en el software.	Media-Alta
HU-14	133	Como desarrollador requiero apoyarme en las tecnologías emergentes para poder garantizar información clara al usuario sobre los resultados obtenidos por los juegos.	Alta
HU-15	138	Como desarrollador requiero que mi espacio de trabajo tenga un orden claro para poder mantener un trabajo ordenado y estable.	Media-Alta
HU-16	142	Como desarrollador requiero que el código este organizado completamente evitando redundancias, cuellos de botella, usando los pilares de la programación que me garanticen que el código sea reutilizable escalable y mantenible facilitando su diseño.	Alta
HU-17	147	Como usuario quiero que el programa no tarde mucho en cambiar de apartados.	Baja

Nota. La tabla muestra en listado las 17 Historias de Usuario actuales del Software junto con las ID de creación en la web Azure DevOps

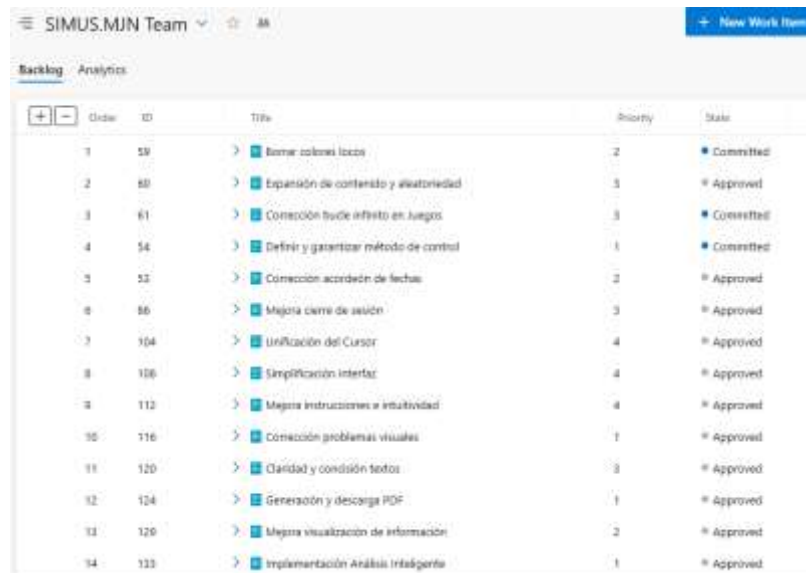
Product Backlog

Tabla 2

Product Backlog

ID	Backlog items	Priority	Estado
59	Borrar colores locos	Media-Alta	Committed
60	Expansión de contenido y aleatoriedad	Media	Approved
61	Corrección bucle infinito en juegos	Media	Committed
54	Definir y garantizar método de control	Alta	Committed
53	Corrección acordeón de fechas	Media-Alta	Approved
86	Mejora cierre de sesión	Media	Approved
104	Unificación del cursor	Baja	Approved
108	simplificación interfaz	Baja	Approved
112	Mejora instrucciones e intuitivita	Baja	Approved
116	Corrección problemas visuales	Alta	Approved
120	Claridad y concisión textos	Media	Approved
124	Generación y descarga PDF	Alta	Approved
129	Mejora visualización de información	Media-Alta	Approved
133	Implementación Análisis inteligente	Alta	Approved
138	Organización profesional carpetas	Media-Alta	Done
142	Optimización estructural código	Alta	Committed
147	Tiempo de respuesta < 3 segundos	Baja	Approved

Nota. En la tabla se puede observar la organización de los Backlogs que se pueden encontrar en Azure DevOps.

Figura 2**Backlogs en Azure DevOps**


Order	ID	Title	Priority	State
1	59	Borrar columnas locas	2	Committed
2	60	Expansión de contenido y aleatoriedad	3	Approved
3	61	Conexión trade infinito en juegos	3	Committed
4	54	Definir y garantizar método de control	1	Committed
5	53	Corrección acordeón de fechas	2	Approved
6	86	Mejora cierre de sesión	3	Approved
7	104	Unificación del Cursor	4	Approved
8	106	Simplificación interfaz	4	Approved
9	112	Mejora instrucciones e intuitividad	4	Approved
10	116	Corrección problemas visuales	1	Approved
11	120	Claridad y cohesión textos	3	Approved
12	124	Generación y descarga PDF	1	Approved
13	129	Mejora visualización de información	2	Approved
14	133	Implementación Análisis Inteligente	1	Approved

Nota. En la Figura se puede evidenciar la organización de Backlogs en Azure DevOps

Sprint

Sprint Actual: Sprint 1

Duración: 3 Semanas

Objetivo:

Depurar el sistema refactorizando el código base mejorando el rendimiento general y la interfaz del usuario, garantizando un entorno solido libre de fallos críticos y técnicos.

Historias de Usuario Seleccionadas: HU-01, HU-03, HU-04, HU-06, HU-10, HU-15, HU-16, HU-17

Actividades Realizadas.

- Conexión local a repositorio PKAN en GitHub
- Conexión de repositorio a Azure DevOps
- Desarrollo completo de la HU-15
- Desarrollo Parcial de las Historias de Usuario: HU-01, HU-03, HU-04, HU-16

Estado del Sprint.

En Desarrollo, se ha concluido la primer Historia de usuario (HU-15), mientras tanto se siguen desarrollando las que nos quedan pendientes, El trabajo se ha establecido de la mejor manera con el repositorio correctamente conectado a nuestros computadores, GitHub y Azure DevOps, se debe mejorar el rendimiento por parte de los desarrolladores dado que se encuentra en la ultima semana de desarrollo y aun no se han iniciado a trabajar las historias de usuario HU-10 y HU-17 respectivamente.

Figura 3

Avance Sprint 1 Azure DevOps



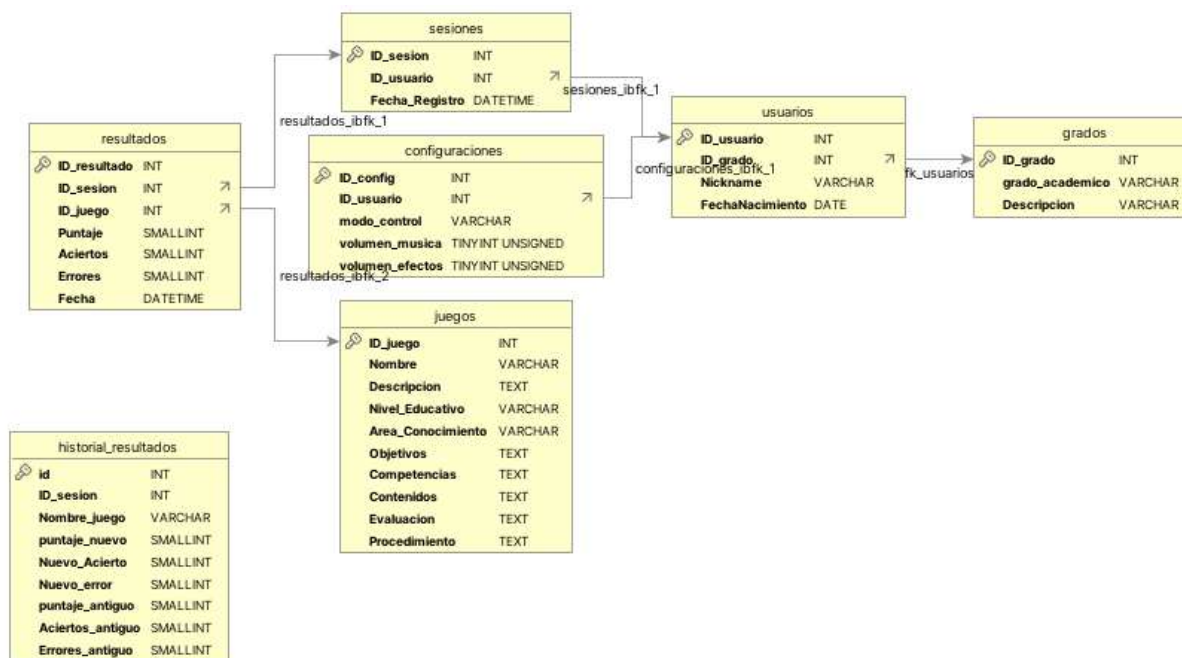
Nota. La figura muestra la interfaz y el progreso de los Sprints en Azure DevOps en escala resumida.

Modelo de base de datos

Modelo entidad relación

Figura 4

Modelo Entidad Relación



Nota. Diagrama Entidad relación de la base de datos Simus_mjn tomado de DbVisualizer

El modelo entidad relación nos permite garantizar el flujo eficiente de la información del software, este modelo es una herramienta conceptual que permite representar de forma gráfica la estructura lógica de la base de datos haciendo más fácil la lectura y comprensión de las interrelaciones de todos los datos.

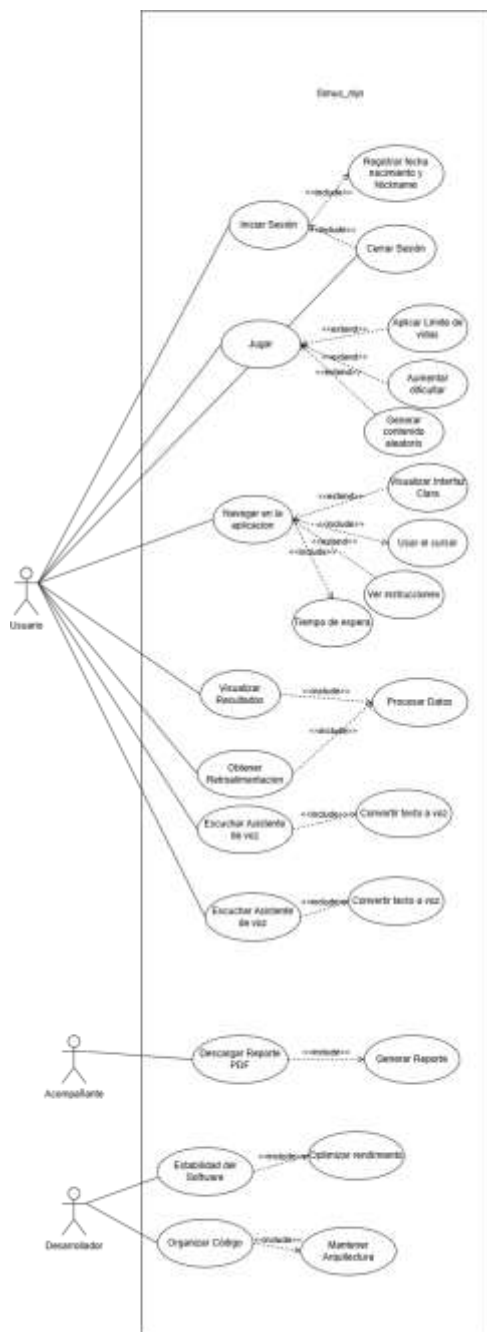
La Figura 4 nos muestra las entidades y sus atributos, estableciendo las conexiones necesarias para el correcto almacenamiento de datos, organizando también la información del software en tablas como historial_resultados la cual por medio de un trigger va almacenando todo el historial de los resultados de la cual luego podemos hacer un análisis del progreso del usuario frente al uso de Simus_mjn.

Diagramas UML

Diagrama de casos de uso

Figura 5

Diagrama Casos de Uso

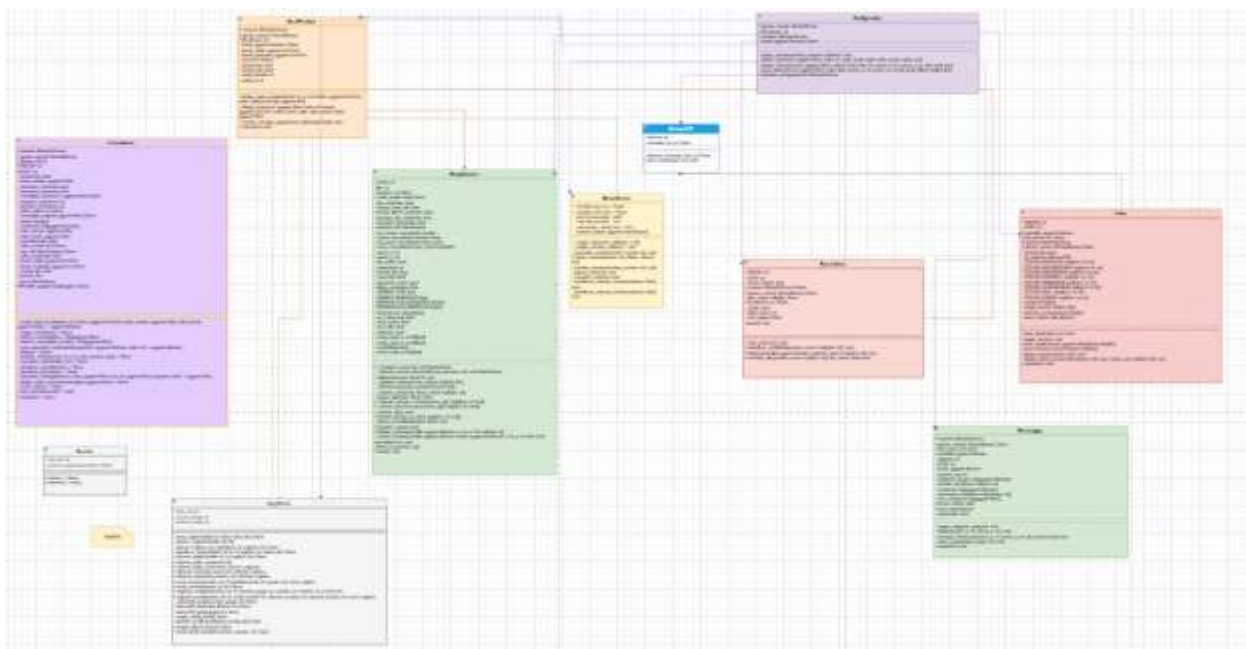


Nota. El diagrama presenta de forma visual los requisitos del sistema Simus_mjn frente a los actores involucrados en el mismo.

Diagrama de Clases

Figura 6

Diagrama de Clases Simus_mjn



Nota. Diagrama de casos de uso del software Simus_mjn hecha en Draw.io.

Desarrollo del Software

12.1 Evidencia del Módulo Funcional

Figura 7.

Inicio de Sesión



Nota. En la presente imagen se muestra el módulo de iniciar sesión.

Figura 8

Módulo de registro



Registro de Usuario

Nick Name:

Fecha Nac.: Día Mes Año

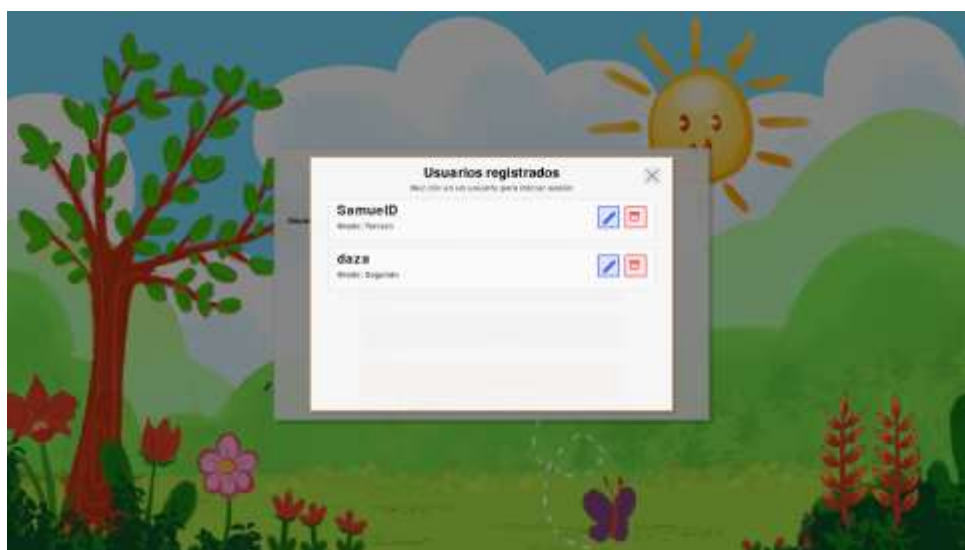
Grado: Seleccione el grado

Registrar

Nota. En esta imagen se puede observar el apartado de registro que da camino al inicio de sesión

Figura 9.

Módulo de Usuarios Registrados



Usuarios registrados

Mostrar solo los usuarios que se han registrado

SamuelD	Grado: Primaria		
diaz	Grado: Secundaria		

Nota. La figura muestra como por medio de la conexión con la base de datos se visualizan los registros de usuarios actuales en la base de datos.

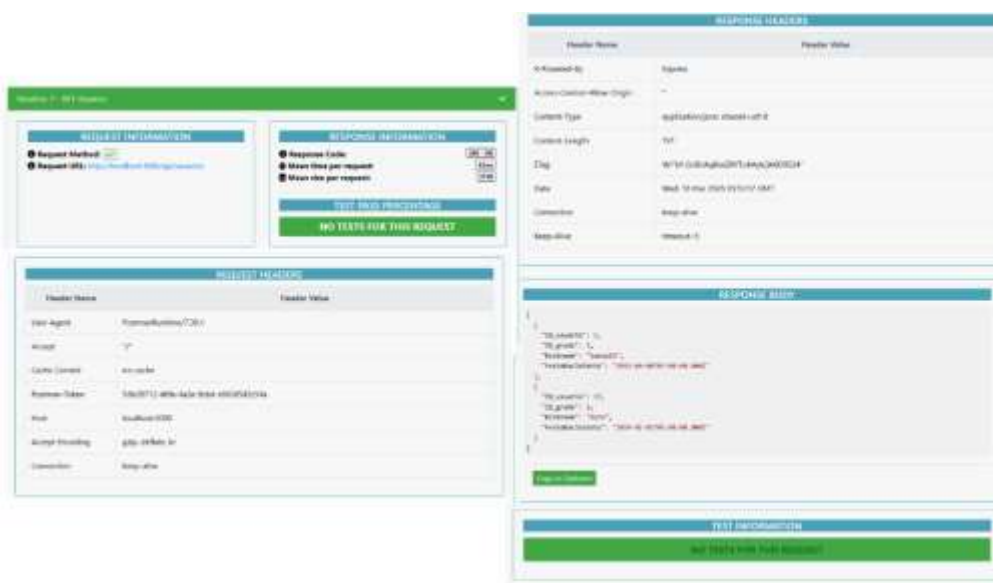
Pruebas Unitarias

Para validar el buen funcionamiento y comunicación entre los componentes del software Simus_mjn, se ocupó la aplicación de escritorio Postman, una plataforma destacada en el desarrollo de pruebas de Interfaces de Programación de Aplicaciones (APIs). Las cuales actúan como un puente de comunicación permitiendo al software enviar y recibir datos de manera estructurada.

Postman hizo simple la ejecución de pruebas sobre los servicios del sistema permitiendo observar el comportamiento de la Api frente a los métodos HTTP fundamentales para obtener datos (GET), crear registros (POST), actualizar la información (PUT), y eliminar registros (DELETE). Estas pruebas nos garantizan que la lógica manejada internamente responda asertivamente a las peticiones de los usuarios, asegurando la integridad del software en cuanto a las transacciones de los datos.

Figura 10

Informe Prueba Unitaria



Nota. La figura muestra los resultados de las pruebas de conexión del API, informe generado mediante Newman el cual es el ejecutor de colecciones por línea de comandos para Postman.

Para hacer la correcta visualización del reporte realizado por Newman de las pruebas unitarias sobre la API mediante Postman, acceda haciendo [clic aquí](#).

Evidencia de instalaciones y registros

Plataforma Cloude Azure DevOps

Figura 11

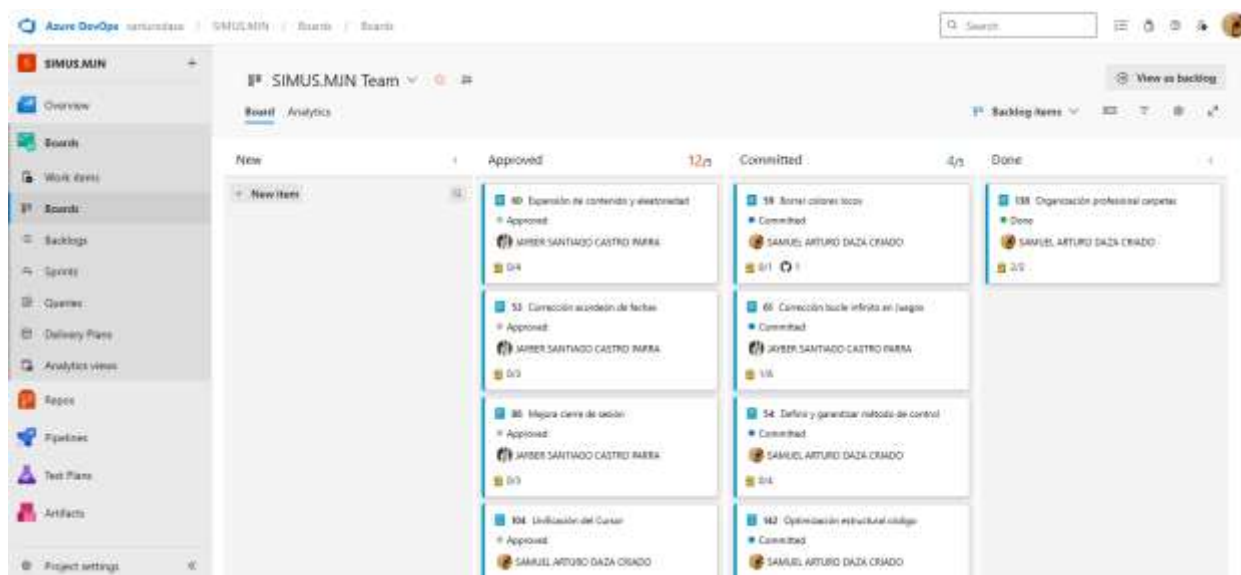
Repositorio Azure DevOps



Nota. En esta figura podemos visualizar los commits realizados a nuestro proyecto.

Figura 12

Board SIMUS.MUN Azure DevOps



Nota. En la presente figura se puede apreciar el tablero principal donde se nos muestra las historias de usuario junto con sus estados actuales.

Figura 13*Conexión Repositorios Azure DevOps y GitHub*

Development

Add link

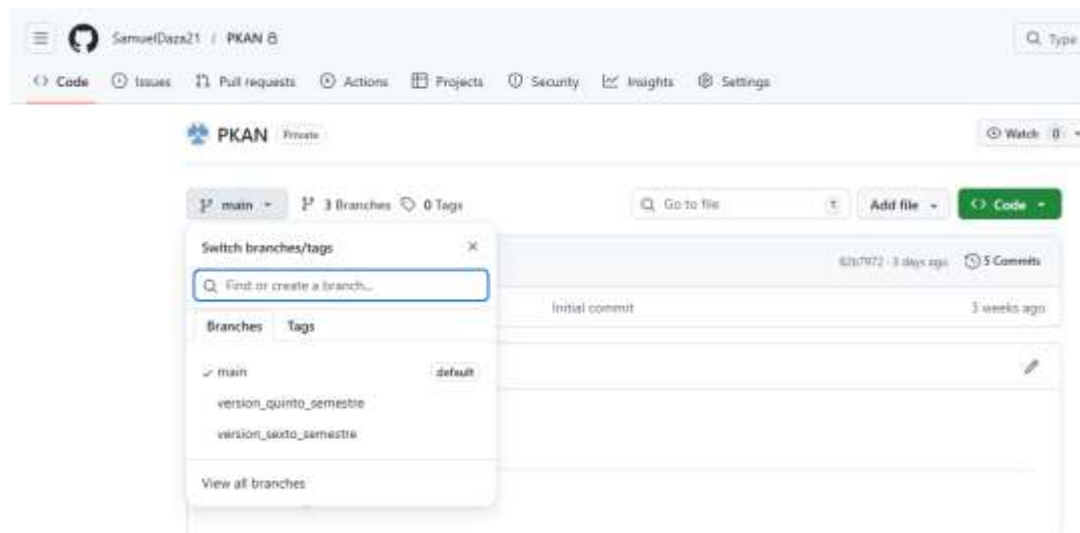
GitHub Commit

  [Probando conexion AB#59](#)
Updated Yesterday

Related Work

Nota. En la Figura podemos observar un commit realizado a fecha del día de ayer 17 de marzo del presente año, commit hecho al backlog Borrar Colores Locos con ID#59

Control de Versiones GitHub

Figura 14*Control de Versiones Repositorio PKAN*

Nota. La figura nos muestra que tenemos 3 ramas, una principal y dos que nos hacen el control de las versiones, de las cuales la versión activa es “versión_sesto_semestre”

Enlaces de Apoyo

Para permitir el seguimiento detallado de lo hablado en el documento se proporcionan tres enlaces le permitirán al lector observar las diferentes fases del ciclo de vida de desarrollo del software Simus_mjn.

Gestión de Proyecto

Azure DevOps plataforma usada para la planificación de Sprint, gestión de tareas y seguimiento de hitos del equipo de desarrollo, recordar aceptar la invitación antes.

Enlace: <https://dev.azure.com/sarturodaza/SIMUS.MJN>

Repositorio en GitHub

Control de entregables del proyecto Simus_mjn donde se aloja archivos como:

- Diagrama de casos de uso
- Diagrama Modelo entidad Relación
- Carpeta Newman de Pruebas Unitarias de Funcionalidad en Postman
- Carpeta de archivos de la Api
- Documento Proyecto de semestre Azure DevOps-GitHub

Enlace: https://github.com/SamuelDaza21/PKAN_entrgables

Conclusiones

1. Se ha logrado abordar un gran porcentaje de los lineamientos establecidos por el docente frente a la entrega.
2. Se logran apreciar con claridad cada uno de los gráficos especificados dentro de la rubrica permitiendo el despliegue estructural del Software.
3. Se logro establecer las conexiones respectivas frente al trabajo remoto del software con el repositorio local, la nube y el GitHub, haciendo posible la organización del trabajo en general.

4. Se logro mostrar un avance pleno del Software Simus_mjn el cual permite dar claridad al rendimiento de su desarrollo con cada uno de los apartados tratados en este documento.
5. El software dado a su trabajo local, dio complejidad para poder crear su ejecutable (.exe) gracias al tamaño de librerías y al ser un proyecto hibrido, es decir su contenido esta creado con Python y JavaScript. Esto genero problemas al tratar de combinar todas las carpetas en una sola y que los archivos fueran compatibles en un solo lugar.
6. La herramienta de Azure DevOps es muy útil y versátil para mostrar archivos desde repositorios, pero en nuestro caso no nos beneficio debido a que, para el uso de pipelines, en el presente caso el uso del mismo este dado por servicio de la Universidad de Cundinamarca, se debieron pedir permisos los cuales son respondidos en promedio de 2 a 4 días hábiles, por lo que para la presente entrega no alcanza a estar listo para su entrega.
7. Las herramientas usadas son versátiles y permiten mantener una buena estructuración y organización del proyecto en cada una de sus etapas.